

Lab No: 05

* Lab Title: **while Loop**

○ Learning objectives:

The following types will be covered in this lab session

- While Loop
- Nested While Loop
- Do-While Loop

➤ Loop

Loops are used to repeat a block of code. Being able to have your program repeatedly execute a block of code is one of the most basic but useful tasks in programming -- many programs or websites that produce extremely complex output are really only executing a single task many times. Now, think about what this means: a loop lets you write a very simple statement to produce a significantly greater result simply by repetition.

➤ While Loop

The most basic loop in C++ is the while loop. A while statement is like a repeating if statement. Like an If statement, if the test condition is true: the statements get executed. The difference is that after the statements have been executed, the test condition is checked again. If it is still true the statements get executed again. This cycle repeats until the test condition evaluates too false.

A **while loop** is a control flow statement that allows code to be executed repeatedly based on a given condition. The *while* consists of a block of code and a condition. The condition is first evaluated and if the condition is true the code within the block is then executed. This repeats until the condition becomes false

- Syntax

Basic syntax of while loop is as follows:

```
while (expression)
```

```
{
```

```
Single statement
```

```
Or
```

```
Block of statements;
```

```
}
```

- The expression can be any combination of Boolean statements that are legal. Even, (while x ==5 || v == 7) which says execute the code while x equals five or while v equals 7.

➤ **Example:**

Write a program using while loop who print the values from 10 to 1.

```
//This program prints 10 to 1 using while loop

#include <iostream.h>
#include <conio.h>
void main ()
{
    clrscr();
    int i=10; //loop control variable
    while (i>0)
    {
        cout<<i<<"\n";
        i=i-1;
    }
    cout<<"outside loop now";
    getch();
}
```

➤ **Example:**

*//Prints the number entered by the user and
exits if the number entered is -1*

```
#include <iostream.h>
#include <conio.h>

void main()
{
    clrscr();
    int num=5;
    while (num!=-1)
```

```
{
    cout<<"Enter a number to print: -1 to quit\n";
    cin>>num;
    cout<<"You entered: "<<num<<"\n";
}

cout<<"outside loop now";
getch();
}
```

- Printing the numbers you entered using a while loop. Exit the loop if the number entered is -1

➤ Nested While Loop:

- A While loop inside another while loop is called nested while loop

➤ **Example:** Write a program that prints the following pattern.

1 2 3 4 5 6 7

1 2 3 4 5 6

1 2 3 4 5

1 2 3 4

1 2 3

1 2

1

```
#include <iostream.h>
#include <conio.h>

void main ()
{
    clrscr ();
    int num=7;
    int j;

    while (num>0)
    {
        j=1;
        while (j<=num) //nested loop
        {
            cout<<j<<"\t";
            j++; //its equivalent to j=j+1;
        }
        cout<<"\n";
        num--;
    }

    cout<<"outside loop now";
    getch();
}
```

- **Its format is:**
do statement while (condition);

Its functionality is exactly the same as the while loop, except that condition in the do-while loop is evaluated after the execution of statement instead of before, granting at least one execution of statement even if condition is never fulfilled.

Lab Tasks

➤ Task 1: While Loop

1. Write a C program that uses a while loop to display numbers from 1 to 10.
2. Display each number on a separate line.

➤ Input:

```
#include <iostream>    // Input aur output functions ke
                        // liye header file
using namespace std;  // std:: likhne ki zarurat nahi

int main()            // Program execution yahan se start hoti hai
{
    int i = 1;        // Counter variable initialize kiya (1
                        // se start)

    // while loop 1 se 10 tak numbers display karega
    while (i <= 10)
    {
        cout << i << endl;    // Har number ko new line
        // par print karega
        i++;                    // Counter ko 1 se increase
        // karta hai
    }

    return 0;          // Program successfully end hota hai
}
```

➤ **Output:**

```
1
2
3
4
5
6
7
8
9
10
```

➤ **Task 2: Do-While Loop**

Repeat the first task using do... while loop.

➤ **Input:**

```
#include <iostream> // Input aur output functions ke liye header file
using namespace std; // std:: likhne ki zarurat nahi

int main() // Program execution yahan se start hoti hai
{
    int i = 1; // Counter variable initialize kiya (1 se start)

    // do...while loop pehle execute hota hai phir condition check hoti
    // hai
    do
    {
        cout << i << endl; // Number ko new line par print karta hai
        i++; // Counter ko 1 se increase karta hai
    }
    while (i <= 10); // Condition check: 10 tak loop chalega

    return 0; // Program successfully end hota hai
}
```

➤ **Output:**



1
2
3
4
5
6
7
8
9
10

➤ **Task 3: Sum using While Loop**

1. Write a C program that prompts the user to enter a positive integer.
2. Use a while loop to calculate the sum of all numbers from 1 to the entered integer.
3. Display the calculated sum.

➤ **Input:**

```
#include <iostream> // Input aur output functions ke liye
header file
using namespace std; // std:: baar baar likhne ki zarurat
nahi

int main() // Program execution yahan se start hoti hai
{
    int num; // User ka input store karne ke liye
    variable

    // User se positive integer input lene ke liye message
    cout << "Enter a positive integer: ";
    cin >> num; // Input lena
```

```
int sum = 0;    // Sum store karne ke liye variable
int i = 1;     // Loop counter initialize

// while loop 1 se num tak numbers ka sum calculate
// karega
while (i <= num)
{
    sum = sum + i;    // Har number ko sum me add karta
    // hai
    i++;              // Counter ko increase karta hai
}

// Final result display
cout << "The sum of all numbers from 1 to "
      << num << " is " << sum << endl;

return 0;    // Program successfully end
}
```

➤ Output:

```
Enter positive integer:
10
The sum is 55
```

➤ Conclusion:

In this program, a `while` loop is used to calculate the sum of all numbers from 1 to the user-entered positive integer. The program first takes input from the user, then continuously adds each number starting from 1 up to the given value. Finally, it displays the total sum as the output. This program helps in understanding the working of loops, especially the `while` loop, in performing repetitive addition tasks.